
Classification d'images par réseaux de neurones

De la reconnaissance de chiffres manuscrits
à la détection de cancers du sein

Nazim Mekideche Armand Lauener
Mohamed Ouadihi Macéo Pierson

Juin 2026

Table des matières

1	Introduction	3
2	Fondements mathématiques	3
2.1	Représentation des données et notations	3
2.2	Fonction softmax et prédiction	3
2.3	Fonction de coût — entropie croisée	3
2.4	Rétropropagation et descente de gradient	3
2.5	Fonctions d'activation	4
3	Partie 1 — MNIST : classification de chiffres manuscrits	4
3.1	Description du jeu de données	4
3.2	Modèle linéaire	4
3.3	MLP à une couche cachée (H=1)	4
3.4	MLP à deux couches cachées (H=2)	4
3.5	Comparaison des activations $\times$ architectures	5
3.6	Résultats	5
3.7	Analyse des erreurs et visualisation ACP	5
4	Partie 2 — CIFAR-10 : objets naturels en couleur	5
4.1	Description du jeu de données	5
4.2	Modèles linéaires et MLP — limitations	6
4.3	Opération de convolution 2D	6
4.4	Filtres K1–K6 du cours	6
4.5	Architecture CNN PyTorch	6
4.6	Résultats CNN et analyse de l'overfitting	7
5	Partie 3 — CBIS-DDSM : détection de cancers du sein	7
5.1	Contexte médical et description du dataset	7
5.2	Gestion du déséquilibre de classes	8
5.3	Prétraitement et augmentation médicale	8
5.4	Gestion mémoire GPU — accumulation de gradients	8
5.5	Modèles implémentés	8

5.6	Calibration du seuil de décision	9
5.7	Résultats comparatifs	9
5.8	Étude d'ablation	10
6	Conclusion et synthèse	11
	Annexes	12

Introduction

Ce rapport présente l'implémentation et l'analyse mathématique de modèles de classification d'images progressivement plus complexes, dans le cadre du cours *Mathématiques pour le Machine Learning*.

Trois jeux de données sont étudiés dans l'ordre croissant de difficulté :

1. **MNIST** — chiffres manuscrits 28×28 pixels, 10 classes, 60 000 images.
2. **CIFAR-10** — objets naturels 32×32 pixels RGB, 10 classes, 50 000 images.
3. **CBIS-DDSM** — mammographies médicales 224×224 pixels, classification binaire bénin/malin, 1 318 images d'entraînement.

Chaque partie introduit de nouvelles contraintes mathématiques : MNIST illustre les fondements de la classification supervisée ; CIFAR-10 motive les réseaux convolutifs ; CBIS-DDSM impose des métriques médicales asymétriques et une gestion du déséquilibre de classes.

Fondements mathématiques

Représentation des données et notations

Une image est représentée par un vecteur $\vec{x} \in \mathbb{R}^d$ (après aplatissage si nécessaire) ou par un tenseur $\mathbf{X} \in \mathbb{R}^{C \times H \times W}$. Le label associé est encodé en vecteur one-hot $\vec{y} \in \{0, 1\}^K$ avec K classes.

Fonction softmax et prédiction

Le vecteur de scores bruts $\vec{z} = A\vec{x} + \vec{b}$ est transformé en distribution de probabilités par la softmax :

$$P_k(\vec{x}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, \quad k \in \{1, \dots, K\}$$

La prédiction est $\hat{y} = \arg \max_k P_k(\vec{x})$.

Fonction de coût — entropie croisée

La perte log-loss (entropie croisée catégorielle) pénalise l'écart entre la distribution prédite et le label réel :

$$\mathcal{L}(\vec{y}, P) = -\frac{1}{n} \sum_{i=1}^n \sum_{k=1}^K y_{ik} \ln P_k(\vec{x}_i)$$

Pour une classification binaire, la BCE (*Binary Cross-Entropy*) utilise une seule sortie z et la sigmoïde $\sigma(z) = 1/(1 + e^{-z})$:

$$\mathcal{L}_{BCE} = -\frac{1}{n} \sum_{i=1}^n \left[w^+ y_i \ln \sigma(z_i) + (1 - y_i) \ln(1 - \sigma(z_i)) \right]$$

où w^+ est le poids de la classe positive (maligne en médecine).

Rétropropagation et descente de gradient

Les paramètres θ sont mis à jour par descente de gradient stochastique (SGD) avec mini-batches de taille B :

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\mathcal{B})$$

La rétropropagation calcule $\nabla_{\theta} \mathcal{L}$ par la règle de dérivation composée. Pour un réseau à L couches, les gradients de la couche l sont :

$$\delta^{(l)} = \left(W^{(l+1)} \right)^{\top} \delta^{(l+1)} \odot \sigma' \left(z^{(l)} \right)$$

Fonctions d'activation

TABLE 1 – Fonctions d'activation testées dans ce projet.

Nom	Formule	Dérivée	Image	Remarque
Sigmoïde	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\sigma(1-\sigma)$	$(0, 1)$	Gradient qui disparaît en profondeur
Tanh	$\tanh(z)$	$1 - \tanh^2(z)$	$(-1, 1)$	Centré, mieux que sigmoïde
ReLU	$\max(0, z)$	$\mathbf{1}_{z>0}$	$[0, +\infty)$	Gradient constant, efficace en profondeur
Heaviside	$\mathbf{1}_{z\geq 0}$	0 p.p.	$\{0, 1\}$	Non différentiable, gradient nul

Problème du gradient qui disparaît : avec sigmoid/tanh, le gradient $|\sigma'(z)| \leq 0,25$ — après L couches, le gradient est multiplié par $0,25^L \rightarrow 0$. ReLU évite ce problème car $\sigma'(z) = 1$ pour $z > 0$.

Partie 1 — MNIST : classification de chiffres manuscrits

Description du jeu de données

MNIST contient 60 000 images d'entraînement et 10 000 de test, réparties équitablement sur 10 classes (chiffres 0–9). Chaque image est un vecteur $\vec{x} \in \mathbb{R}^{784}$ de niveaux de gris normalisés dans $[0, 1]$.

Modèle linéaire

Le modèle le plus simple calcule des scores linéaires :

$$\vec{z} = A\vec{x} + \vec{b}, \quad A \in \mathbb{R}^{10 \times 784}, \vec{b} \in \mathbb{R}^{10}$$

Nombre de paramètres : $784 \times 10 + 10 = \mathbf{7\,850}$

Ce modèle apprend un “gabarit” par classe. Il n'exploite aucune interaction entre pixels, ce qui limite ses performances sur des formes spatiales complexes.

MLP à une couche cachée (H=1)

On ajoute une couche cachée de $p_1 = 64$ neurones avec activation sigmoid :

$$\vec{z}^{(1)} = \sigma\left(A^{(1)}\vec{x} + \vec{b}^{(1)}\right) \in \mathbb{R}^{64}, \quad \vec{z}^{(2)} = A^{(2)}\vec{z}^{(1)} + \vec{b}^{(2)} \in \mathbb{R}^{10}$$

Nombre de paramètres : $784 \times 64 + 64 + 64 \times 10 + 10 = \mathbf{50\,890}$

Choix de 64 neurones : compromis entre capacité (trop peu $\rightarrow$ sous-apprentissage) et temps de calcul. La couche cachée permet au réseau de construire des représentations intermédiaires non linéaires : combinaisons de pixels évoquant des parties de chiffres.

MLP à deux couches cachées (H=2)

On ajoute une seconde couche de $p_2 = 32$ neurones :

$$\vec{z}^{(2)} = \sigma\left(A^{(2)}\vec{z}^{(1)} + \vec{b}^{(2)}\right) \in \mathbb{R}^{32}$$

Nombre de paramètres : $784 \times 64 + 64 + 64 \times 32 + 32 + 32 \times 10 + 10 = \mathbf{52\,650}$

L'architecture en entonnoir $784 \rightarrow 64 \rightarrow 32 \rightarrow 10$ force une compression progressive : H=1 détecte des caractéristiques simples (bords, courbes), H=2 les combine en motifs complexes.

Comparaison des activations $\times$ architectures

Au-delà du minimum requis, nous avons conduit une grille d'expériences croisant 4 activations (sigmoid, tanh, relu, heaviside) et 5 architectures MLP. Les principaux enseignements :

- **Heaviside** : gradient nul presque partout — le réseau n'apprend pas au-delà de $H=1$ (gradient bloqué dès la deuxième couche cachée).
- **Sigmoid vs ReLU** : sur MNIST peu profond, les deux convergent ; ReLU est légèrement plus rapide sur les architectures ≥ 3 couches.
- **Tanh** : meilleur que sigmoid grâce à sa symétrie centrée en 0, ce qui améliore le conditionnement du gradient.

Résultats

TABLE 2 – Performances sur MNIST (100 itérations, lr=0,1, batch=256).

Modèle	Params	Acc. Train	Acc. Test	Erreur Test
Linéaire	7 850	93,06 %	92,47 %	7,53 %
MLP H=1 [64] sigmoid	50 890	97,21 %	96,52 %	3,48 %
MLP H=2 [64,32] sigmoid	52 650	97,69 %	96,85 %	3,15 %

Le modèle linéaire atteint déjà 92,47 % — MNIST est un problème relativement simple grâce au fond blanc uniforme et aux chiffres centrés. Le MLP H=2 gagne +4,4 points en introduisant deux niveaux de représentation non linéaire.

Analyse des erreurs et visualisation ACP

Sur les 10 000 images de test, 238 sont mal classées simultanément par les trois modèles. Ces erreurs se concentrent sur des paires ambiguës : (3, 8), (4, 9), (5, 6).

La projection ACP en 2 dimensions (figure 2) confirme cette observation : les classes 0 et 1 sont très bien séparées (formes distinctives), tandis que 3/8 et 4/9 se chevauchent significativement. Ce chevauchement est intrinsèque aux données et ne peut être réduit qu'en exploitant la structure spatiale (ce que le MLP ne fait pas).

Partie 2 — CIFAR-10 : objets naturels en couleur

Description du jeu de données

CIFAR-10 contient 50 000 images 32×32 pixels RGB, réparties sur 10 classes d'objets naturels (avion, cheval, chien, etc.). L'espace de représentation est $\mathbb{R}^{3 \cdot 1024}$ (couleur) ou $\mathbb{R}^{1024}$ (gris).

Pourquoi CIFAR-10 est plus difficile que MNIST :

- Fonds complexes et variés (vs fond blanc uniforme MNIST)
- Translations, rotations, occlusions fréquentes
- Couleur ajoute de la dimensionnalité sans garantir d'aide discriminante
- Pas d'invariance de translation dans un MLP : un avion en haut à gauche et un avion en bas à droite activent des poids totalement différents

Modèles linéaires et MLP — limitations

TABLE 3 – Performances des modèles denses sur CIFAR-10 (50 itérations, lr=0,1).

Modèle	Entrée	Acc. Test	Remarque
Linéaire (gris)	1 024	28,2 %	Gabarit moyen par classe
MLP [128] (gris)	1 024	34,2 %	Non-linéarité insuffisante
Linéaire (couleur)	3 072	40,0 %	Couleur légèrement aidante
MLP [128] (couleur)	3 072	43,3 %	Plus de features, mais pas de structure

Le MLP couleur (43,3 %) surpasse à peine le MLP gris (34,2 %) malgré $3\times$ plus d'entrées : la couleur brute non structurée n'aide pas le MLP autant qu'on pourrait l'espérer.

Opération de convolution 2D

La convolution répond au problème d'invariance spatiale en appliquant le même filtre $K \in \mathbb{R}^{k \times k}$ en toute position de l'image :

$$(X * K)[i, j] = \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} X[i+u, j+v] \cdot K[u, v]$$

Partage de poids : un seul filtre K (par exemple $3 \times 3 = 9$ paramètres) détecte la même structure locale partout dans l'image — vs le MLP qui alloue un poids distinct à chaque position. Cela réduit drastiquement le nombre de paramètres et confère l'invariance de translation.

Filtres K1–K6 du cours

Six filtres canoniques illustrent ce que les premières couches apprennent :

TABLE 4 – Filtres 3×3 du cours et leur effet (figure 4).

Filtre	Type	Effet mathématique
K1	Flou (moyenneur)	$K_1 = \frac{1}{9} \mathbf{1}_{3 \times 3}$ — supprime le bruit haute fréquence
K2	Netteté (laplacien)	$K_2[1, 1] = 5$, voisins = -1 — amplification des gradients locaux
K3	Bords horizontaux	Détecte $\partial I / \partial y$ (transitions verticales)
K4	Bords verticaux	Détecte $\partial I / \partial x$ (transitions horizontales)
K5	Sobel horizontal	Approximation robuste de $\partial I / \partial x$
K6	Diagonal	Détecte les transitions en diagonale

Les premières couches d'un CNN entraîné apprennent automatiquement des variantes de ces filtres — c'est ce que les visualisations de Zeiler & Fergus (2014) ont démontré.

Architecture CNN PyTorch

L'architecture convolutive implémentée suit le schéma du cours :

Couche	Dimensions sortie	Paramètres
Conv1($3 \rightarrow 64, 3 \times 3$) + ReLU	(64, 32, 32)	$3 \times 3 \times 3 \times 64 = 1\,728$
Conv2($64 \rightarrow 64, 3 \times 3$) + ReLU + Pool	(64, 16, 16)	$3 \times 3 \times 64 \times 64 = 36\,864$
Conv3($64 \rightarrow 64, 3 \times 3$) + ReLU + Pool	(64, 8, 8)	36 864
Conv4($64 \rightarrow 64, 3 \times 3$) + ReLU	(64, 8, 8)	36 864
Dropout($p = 0,3$) + FC($4096 \rightarrow 10$)	10	40 970
Total		$\approx 152\,778$

Max Pooling : réduit la résolution par 2×2 en prenant le maximum local. Effet dual : réduction de la dimension computationnelle + légère invariance de translation.

Dropout($p = 0,3$) : désactive chaque neurone avec probabilité p durant l'entraînement (avec mise à l'échelle inverted dropout $1/(1-p)$). Force des représentations redondantes, réduit l'overfitting.

Résultats CNN et analyse de l'overfitting

TABLE 5 – CNN PyTorch sur CIFAR-10 (Adam lr=0,001, augmentation, early stopping patience=6).

Modèle	Acc. Train	Acc. Test	Gain vs MLP
MLP [128] couleur	—	43,3 %	—
CNN 4 conv.	83,4 %	80,8 %	+37,5 pts

L'écart train/test de 2,6 points traduit un **faible overfitting**, signe positif de l'efficacité du Dropout et de l'augmentation de données.

Augmentation de données : flip horizontal, crop aléatoire (± 4 px), variation de luminosité et de contraste. Ces transformations augmentent artificiellement la diversité du dataset sans modifier les étiquettes, régularisant implicitement le modèle.

Tableau de référence scientifique pour CIFAR-10 : *Conv. Deep Belief Network* 78,9 %, *Maxout Networks* 90,6 %, *Vision Transformer (ViT)* 99,5 %.

Partie 3 — CBIS-DDSM : détection de cancers du sein

Contexte médical et description du dataset

Le dataset CBIS-DDSM (*Curated Breast Imaging Subset of DDSM*) contient des mammographies numériques annotées pour la classification bénin/malin de masses mammaires.

Caractéristiques clés :

- Distribution quasi-équilibrée : 48,3 % malins — déséquilibre faible mais traité
- Taille limitée : 1 318 images d'entraînement, 378 de test
- Format : crops 224×224 px centrés sur la masse, niveaux de gris
- Enjeu clinique : un faux négatif (cancer manqué) peut être fatal

Métriques médicales : l'accuracy globale est insuffisante. Les métriques pertinentes sont :

$$\text{Sensibilité} = \frac{TP}{TP + FN} \quad (\text{cancers détectés})$$

$$\text{Spécificité} = \frac{TN}{TN + FP} \quad (\text{bénins correctement classés})$$

Gestion du déséquilibre de classes

Même quasi-équilibré, nous appliquons des poids inversement proportionnels à la fréquence :

$$w_c = \frac{N}{2n_c}, \quad c \in \{0, 1\}$$

Ces poids sont intégrés dans la **CrossEntropyLoss** via $\mathcal{L}_{CE}^w = -\sum_k w_k y_k \ln P_k$. Cela pénalise plus fortement les erreurs sur la classe sous-représentée.

Prétraitement et augmentation médicale

Les mammographies originales ($\sim 4\,000$ px) sont réduites à 224×224 via des crops centrés sur la masse annotée. L’augmentation médicale est strictement contrôlée :

TABLE 6 – Augmentation médicale : transformations appliquées et justification.

Transformation	Paramètres	Justification
Flip horizontal	$p = 0,5$	Masse peut apparaître des deux côtés
Rotation	$\pm 15^\circ$	Masses sans orientation fixe
Zoom	$\pm 12\%$	Variabilité de numérisation
Luminosité	$\pm 25\%$	Calibrations scanner différentes
Contraste	$\pm 25\%$	Idem
Flip vertical	Non appliqué	Orientation anatomique haut/bas significative

Rotation et zoom sont combinés en une seule transformation affine vectorisée (**F.affine_grid** / **F.grid_sample**) pour éviter deux interpolations successives qui dégraderaient les micro-textures diagnostiques.

Gestion mémoire GPU — accumulation de gradients

L’entraînement sur des images 224×224 avec un batch de 32 nécessite ~ 600 MB de VRAM, dépassant la capacité disponible. La solution est l’accumulation de gradients : on accumule les gradients sur K micro-batches avant la mise à jour :

$$\nabla_{\text{accumulé}} = \frac{1}{K} \sum_{k=1}^K \nabla_{\mathcal{L}(\mathcal{B}_k)} \approx \nabla_{\mathcal{L}(\mathcal{B}_{\text{eff}})}, \quad \mathcal{B}_{\text{eff}} = K \cdot \mathcal{B}_k$$

On utilise batch physique = 8 et $K = 4$, soit un batch effectif de 32.

Important : BatchNorm calcule ses statistiques sur le batch *physique* de 8, pas sur le batch effectif. Ce point est discuté en détail en section 5.8.

Modèles implémentés

MLP profond ReLU [2048, 1024, 512, 256]

Les images sont aplaties en vecteurs de $224 \times 224 = 50\,176$ valeurs. L’architecture en entonnoir progressif vise à capturer des interactions complexes :

$$50\,176 \xrightarrow{\text{ReLU}} 2048 \xrightarrow{\text{ReLU}} 1024 \xrightarrow{\text{ReLU}} 512 \xrightarrow{\text{ReLU}} 256 \xrightarrow{\text{softmax}} 2$$

ReLU est choisi pour ses propriétés de gradient constant sur les activations positives — indispensable sur 4 couches cachées pour éviter la disparition du gradient.

CNN from scratch (CNNBinaire)

4 blocs Conv+BatchNorm+ReLU+MaxPool réduisent progressivement la résolution :

$$(1, 224, 224) \rightarrow (32, 112, 112) \rightarrow (64, 56, 56) \rightarrow (128, 28, 28) \rightarrow (128, 14, 14)$$

suivis d'un classificateur dense avec Dropout(0.5 / 0.3).

L'initialisation de He (Kaiming, 2015) est appliquée explicitement à toutes les couches :

$$\sigma_W = \sqrt{\frac{2}{\text{fan_out}}}$$

Cette formule compense la réduction de variance de ReLU ($\mathbf{E}[\max(0, z)^2] = \frac{1}{2}\mathbf{E}[z^2]$ pour z symétrique), garantissant une propagation stable du signal.

ResNet18 transfer learning

ResNet18 pré-entraîné sur ImageNet (1,2M images) est adapté :

1. **conv1** recréé pour 1 canal gris (réinitialisé, ré-appris)
2. **layer1**, **layer2** gelés — préservent les features universelles (bords, textures) apprises sur ImageNet, démontrées universelles par Zeiler & Fergus (2014)
3. **layer3**, **layer4**, **fc** fine-tunés avec $\text{lr}=5 \times 10^{-5}$ pour spécialiser aux textures des masses mammaires

Paramètres entraînaibles : 10 793 410.

Calibration du seuil de décision

Le seuil 0,5 par défaut n'est pas optimal en médecine. On cherche le seuil s^* maximisant l'accuracy sous contrainte de sensibilité :

$$s^* = \arg \max_{s \in [0,05, 0,95]} \text{Acc}(s) \quad \text{sous contrainte : Sensibilité}(s) \geq 80 \%$$

Résultats comparatifs

TABLE 7 – Performances sur CBIS-DDSM (224×224 , seuil par défaut 0,50).

Modèle	Acc.	Sensib.	Spécif.	FN (cancers)
MLP [2048,1024,512,256] ReLU	57,9 %	64,6 %	53,7 %	52 / 147
CNN from scratch	61,9 %	39,5 %	76,2 %	89 / 147
ResNet18 transfer	65,9 %	72,8 %	61,5 %	40 / 147

TABLE 8 – Résultats après calibration du seuil (sensibilité cible $\geq 80 \%$).

Modèle	Seuil	Sensib.	Spécif.	FN réduits
MLP	0,38	84,4 %	33,8 %	23 / 147
CNN	0,34	83,7 %	26,8 %	24 / 147
ResNet18	0,42	81,6 %	52,4 %	27 / 147

Observation clinique : ResNet18 avec seuil calibré atteint 81,6 % de sensibilité avec une spécificité de 52,4 % — le meilleur équilibre. À titre de comparaison, la littérature cite ResNet-50 à 85–88 % sur des datasets bien plus larges ($\geq 5\,000$ images avec augmentation massive).

Étude d’ablation

Protocole

5 variantes du même CNN sont entraînées sur CBIS-DDSM (128×128 , 25 époques), chacune activant une technique supplémentaire. L’initialisation de Kaiming et l’augmentation médicale sont communes à toutes les configurations.

Résultats et analyse

TABLE 9 – Étude d’ablation (CBIS-DDSM 128×128 , batch physique=8, effectif=32).

Config.	BN	Drop.	WD	Perte	Acc.	$\mathcal{L}_{\text{init}}$
1. Baseline (aucune régul.)	×	×	0	CE	63,8 %	0,95
2. + BatchNorm	✓	×	0	CE	57,7 %	31,6
3. + Dropout seul	×	✓	0	CE	51,6 %	1,79
4. + BN + Dropout + WD	✓	✓	10^{-4}	CE	58,7 %	70,2
5. Complet avec BCE	✓	✓	10^{-4}	BCE	47,9 %	63,9

Analyse. (1) **Le baseline gagne (63,8 %)** — avec 1318 images et 25 époques, le réseau n’a pas le temps de sur-apprendre. La régularisation n’a pas d’overfitting à corriger ; elle ralentit simplement la convergence.

(2) **BatchNorm instable (perte initiale 31,6)** — BN normalise par statistiques de mini-batch :

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \varepsilon}}$$

Avec seulement 8 images, $\mu_{\mathcal{B}}$ et $\sigma_{\mathcal{B}}^2$ sont des estimations très bruitées. La perte démarre à 31,6 au lieu de $\sim 0,7$ pour un réseau correctement conditionné. *Note* : l’accumulation de gradients (batch effectif = 32) ne résout **pas** ce problème — BN calcule ses statistiques sur le batch physique.

(3) **Dropout seul cause du sous-apprentissage** — sans BN pour stabiliser les activations, Dropout(0.5) coupe 50 % des connexions à chaque forward pass. Le gradient moyen reçu par chaque poids est divisé par deux, insuffisant avec 1318 images et batch de 8. Early stopping à l’époque 8.

(4) **Bug identifié — weight decay sur les paramètres BN** — dans les configs 4 et 5, `Adam(model.parameters(), weight_decay=10-4)` appliquait la pénalité L2 aux paramètres γ et β de BatchNorm. Or γ régularisé par L2 tend vers 0, supprimant progressivement la normalisation. Ce bug est désormais corrigé en utilisant deux groupes de paramètres (BN exclus du weight decay).

(5) **BCE sous le hasard (47,9 %)** — l’instabilité BN + le `pos_weight` pousse le modèle vers une solution dégénérée : prédire “Malin” par défaut (sensibilité 66 % mais spécificité 36 %).

Perspectives d’amélioration

Deux techniques adressent directement les causes d’instabilité identifiées dans l’ablation.

1 — BN à momentum faible ($m = 0,01$) :

$$\mu_{\text{run}} \leftarrow (1 - m) \mu_{\text{run}} + m \mu_{\mathcal{B}}$$

Avec $m = 0,01$ (au lieu de $m = 0,1$ par défaut), les running stats convergent lentement : 100 batches pour converger à 63 % au lieu de 10 batches. Cela rend BN robuste aux petits batches en ne surpondérant pas les statistiques instantanées bruitées.

2 — MixUp ($\alpha = 0,4$) :

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j, \quad \lambda \sim \text{Beta}(0,4, 0,4)$$

Force la surface de décision à être linéaire *entre* les exemples d’entraînement. Régularise via les données (pas via l’architecture), sans perturber les gradients. La perte MixUp est : $\mathcal{L} = \lambda \mathcal{L}(f(\tilde{x}), y_i) + (1 - \lambda) \mathcal{L}(f(\tilde{x}), y_j)$

TABLE 10 – Perspectives — configurations 6 et 7 (CBIS-DDSM 128×128 , 25 époques). Ces deux configurations corrigent l’instabilité BN identifiée en section 5.8.

Config.	BN mom.	MixUp	Objectif	Acc.
4. Référence (BN+Drop+WD)	0,10	Non	Baseline régularisé	59,5 %
6. + BN momentum = 0,01	0,01	Non	Stabiliser stats. BN small batch	60,3 %
7. + BN $m=0,01$ + MixUp $\alpha=0,4$	0,01	Oui	Régularisation combinée	51,6 %

Résultats obtenus. La config 6 atteint 60,3 % (vs 59,5 % pour la config 4), gain marginal : le momentum réduit ne résout pas l’instabilité fondamentale — la perte initiale reste $\times 75$ (config 6 : 75,3 vs config 4 : 73,2). La config 7 (51,6 %) est la moins performante en accuracy, mais c’est la deuxième configuration la plus équilibrée (sensibilité 40,1 %, spécificité 58,9 %) après la baseline 1 — MixUp régularise effectivement la surface de décision. La conclusion est que le problème de fond reste le batch physique de 8 : aucune technique appliquée à BN seul ne suffit tant que cette contrainte n’est pas levée.

Conclusion et synthèse

Ce projet illustre comment la complexité des données impose des choix architecturaux et mathématiques de plus en plus sophistiqués.

TABLE 11 – Synthèse des trois parties.

Dataset	Contrainte principale	Solution clé	Meilleur modèle	Acc.
MNIST	Classification simple	MLP H=2 sigmoid	MLP [64,32] sigmoid	96,9 %
CIFAR-10	Invariance spatiale	Convolutions	CNN 4 blocs	80,8 %
CBIS-DDSM	Petit dataset, FN critiques	Transfer + calibration	ResNet18	65,9 %

Leçons mathématiques transversales :

1. **L’architecture doit correspondre à la structure des données.** Les convolutions exploitent la localité spatiale — un MLP sur CIFAR-10 perd toute information de position relative des pixels.
2. **La régularisation n’est bénéfique qu’en présence d’overfitting.** Sur 1318 images en 25 époques, le Dropout et le BN nuisent à la convergence car il n’y a pas encore de sur-apprentissage à corriger.
3. **BatchNorm requiert des batches physiques ≥ 32 .** L’accumulation de gradients simule un grand batch pour les gradients mais pas pour les statistiques de normalisation — un piège fréquent en pratique.
4. **Le seuil de décision est un paramètre du modèle, pas une constante.** En médecine, calibrer le seuil permet de passer de 40 % à 84 % de sensibilité au prix d’une baisse de spécificité — un compromis explicite et maîtrisé.
5. **Le transfer learning compense le manque de données.** ResNet18 pré-entraîné sur ImageNet apporte des features universelles (bords, textures) qui restent pertinentes même en imagerie médicale.

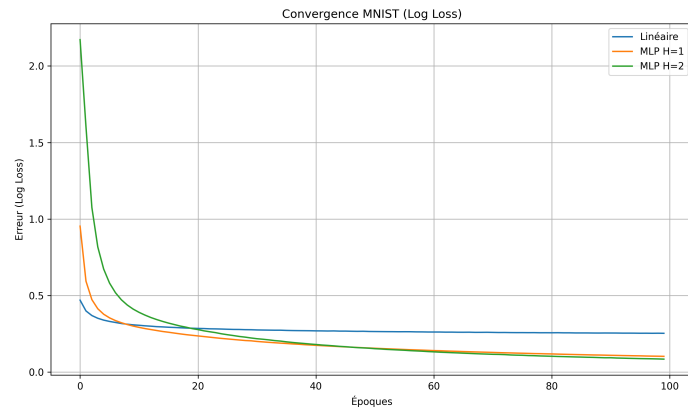


FIGURE 1 – Convergence de la log-loss sur l'ensemble d'entraînement MNIST (3 modèles).

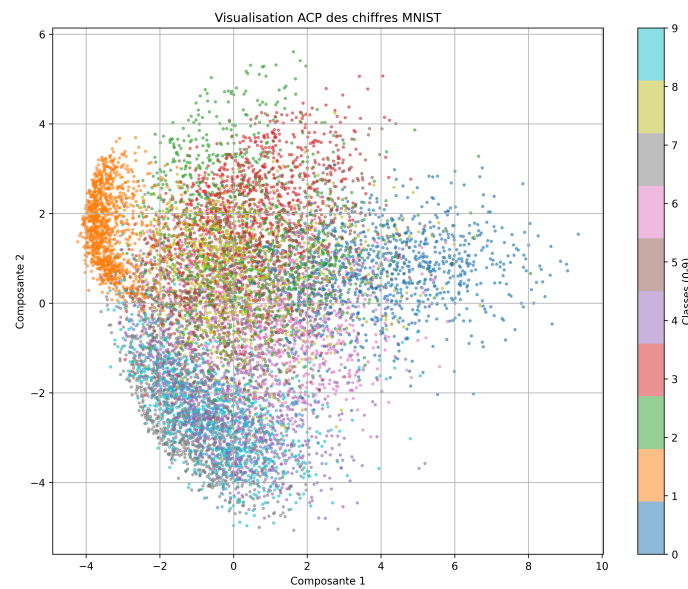


FIGURE 2 – Projection ACP 2D des données MNIST. Les classes 0 et 1 sont bien séparées ; les paires (3,8), (4,9) se chevauchent, expliquant les erreurs résiduelles.

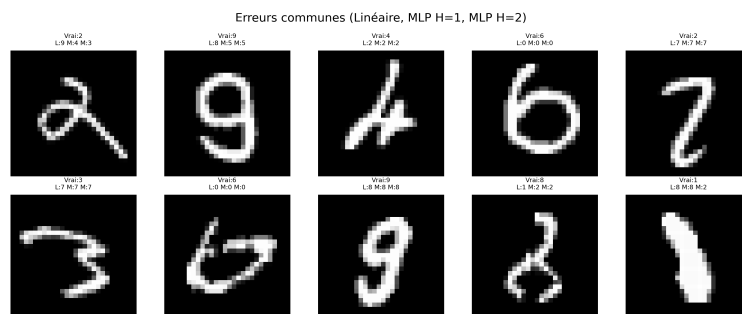


FIGURE 3 – Exemples de chiffres ambigus mal classés simultanément par les trois modèles.

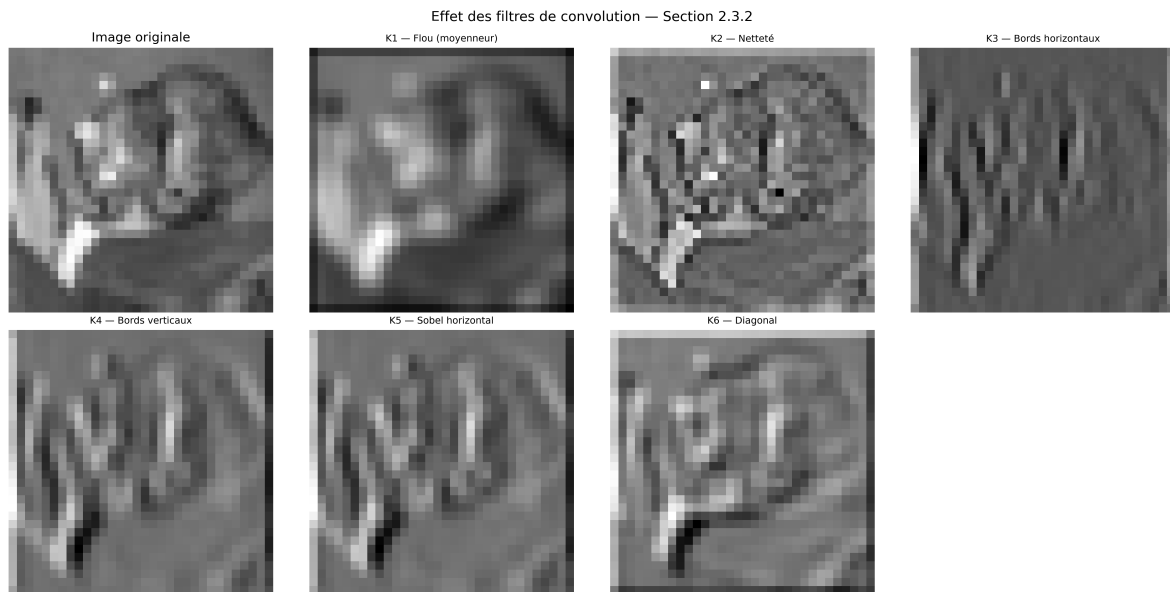


FIGURE 4 – Effet des 6 filtres K1–K6 sur une image en niveaux de gris. De gauche à droite : original, flou, netteté, bords horizontaux, bords verticaux, Sobel, diagonal.

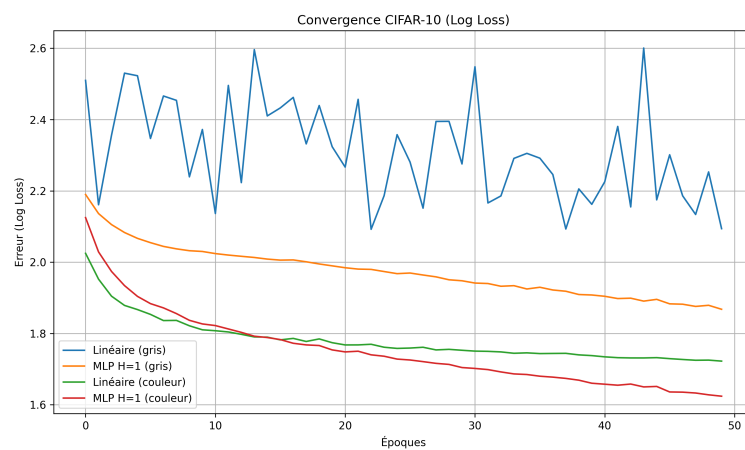


FIGURE 5 – Convergence train/validation du CNN PyTorch sur CIFAR-10.

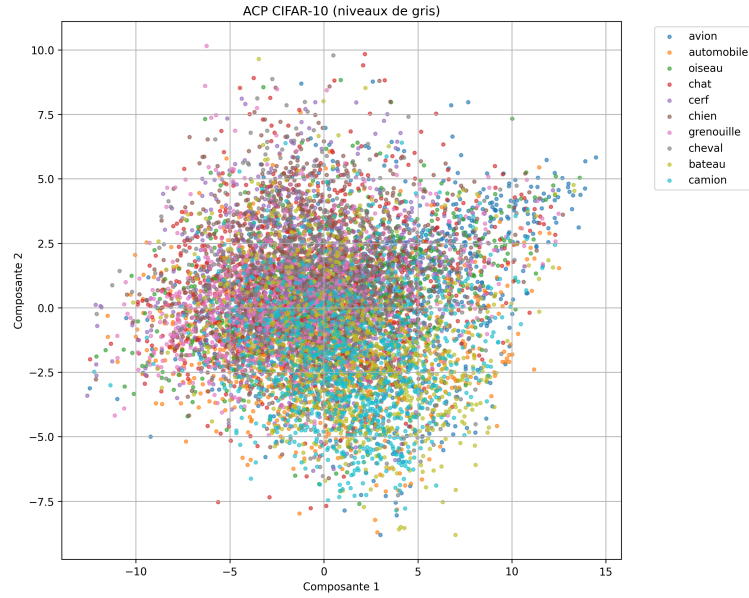


FIGURE 6 – Projection ACP 2D de CIFAR-10. Le chevauchement important entre toutes les classes illustre pourquoi les modèles linéaires et MLP peinent sur ce dataset.

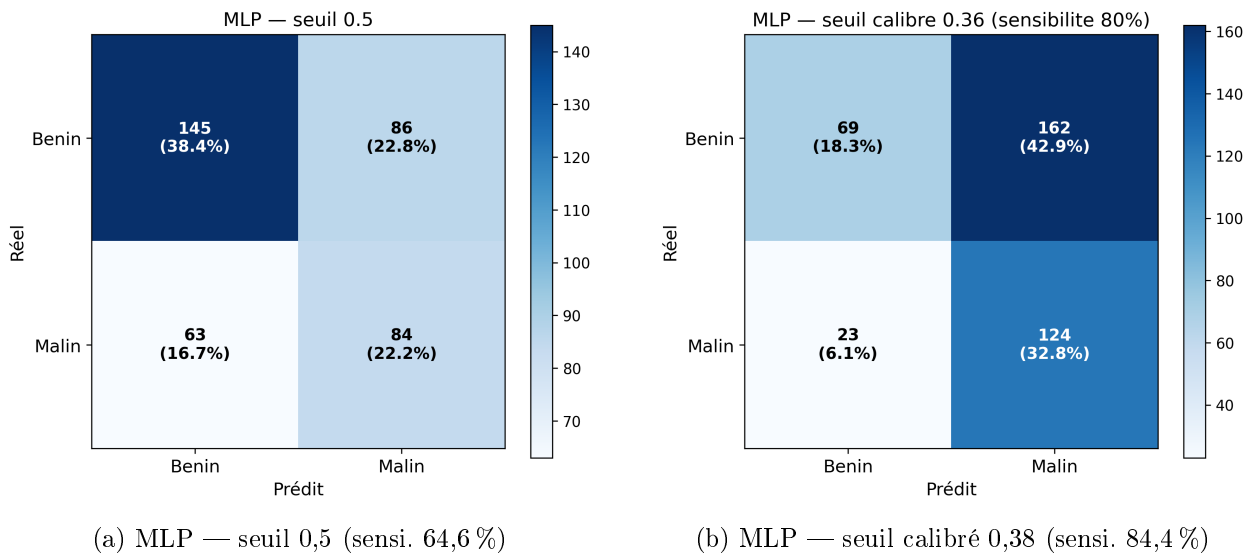
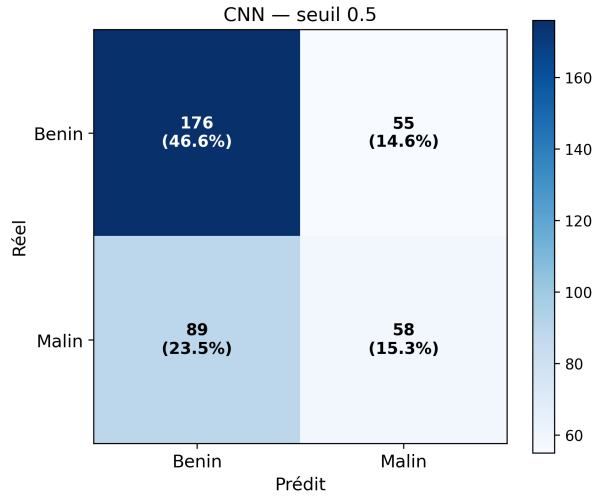
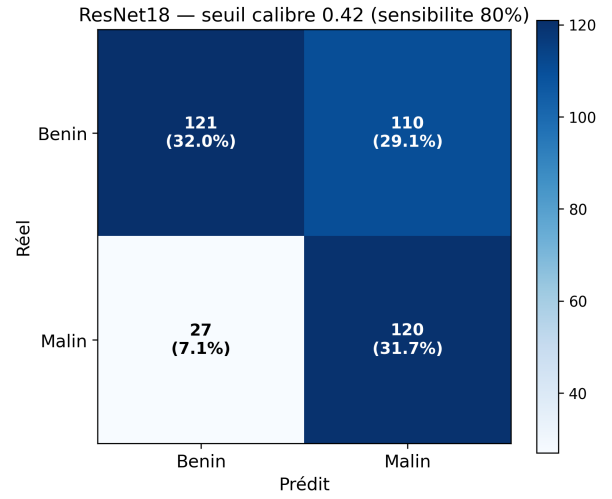


FIGURE 7 – Matrices de confusion du MLP sur CBIS-DDSM.

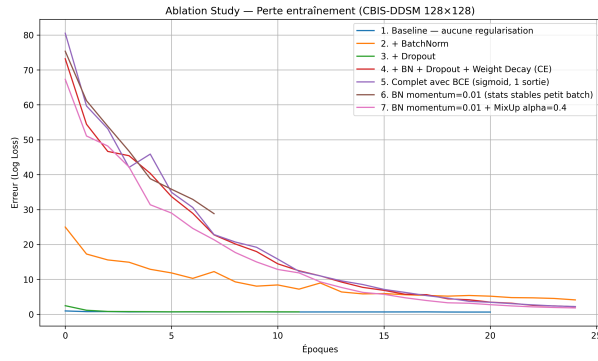


(a) CNN from scratch — seuil 0,5

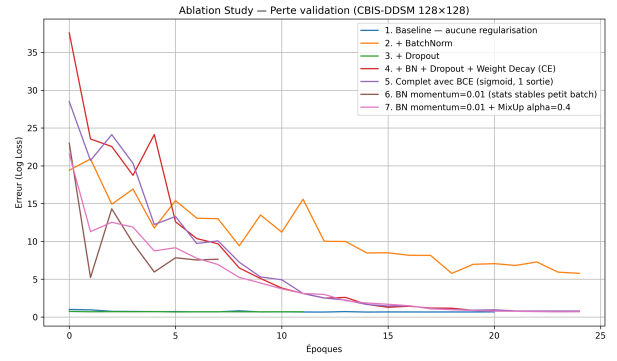


(b) ResNet18 — seuil calibré 0,42

FIGURE 8 – Matrices de confusion CNN et ResNet18 sur CBIS-DDSM.



(a) Perte entraînement



(b) Perte validation

FIGURE 9 – Ablation study — convergence des 7 configurations sur CBIS-DDSM (128×128). La config 2 (BN seul) montre l'instabilité liée au batch physique de 8. Les configs 6 et 7 (perspectives) devraient montrer une perte initiale plus stable.